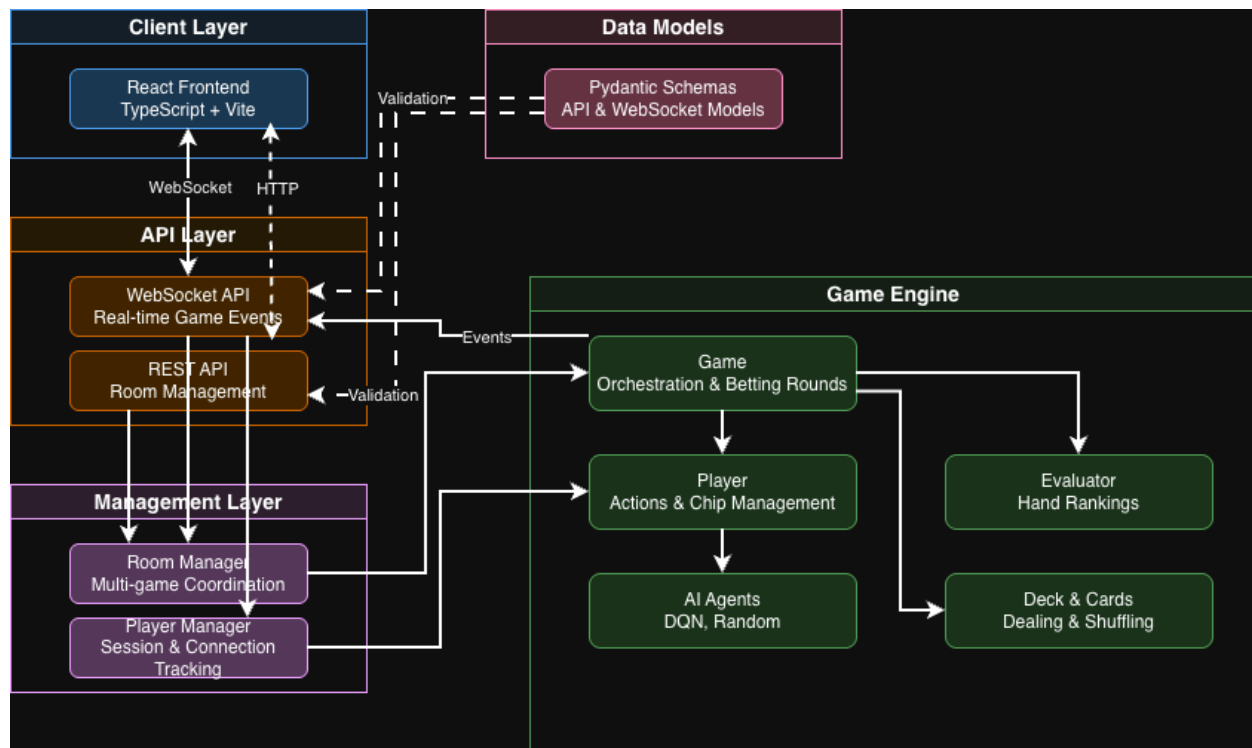


5.0 Requirements Specification

5.1 Introduction

This section of the document specifies the functional and performance requirements for Poker Face 2. This project is a web-based Texas Hold'em Poker application that pits human players against advanced AI agents in real-time. The system migrates from a desktop environment to a web-based architecture, utilizing a FastAPI backend for state management and AI inference, and a React and Vite frontend for the user interface.



The remainder of this document is structured as follows. Section 5.2 contains the Functional Requirements, detailing the breakdown of Computer Software Components (CSCs) for lobby management, game logic, user interface, and AI behavior. Section 5.3 contains the Performance Requirements, outlining the necessary response times, stability, and concurrency standards for the application. Section 5.4 contains the Environment Requirements, specifying the hardware and software stack required for the development and execution of the system.

5.1.1 Assumptions & Constraints

A-1 The system supports a maximum of 6 players per game room.

A-2 The application uses virtual chips only; no real-money gambling is supported.

A-3 Users must be authenticated via a session or token-based system before joining a room.

A-4 The system assumes players have a stable internet connection for real-time WebSocket communication.

A-5 AI agents are preloaded at server startup and available for immediate inference.

A-6 Each player may only occupy one seat per room at a time.

5.2 Functional Requirements

5.2.1 Lobby & Room Management

F-1.1 As a player, I want to retrieve a list of active game rooms via a REST API call so that I can see which tables are currently available to join.

F-1.2 As a player, I want to create a new game room with specific configurations so that I can start a private or new public game if existing ones are full.

F-1.3 As a player, I want to join a specific room via WebSocket using a unique Room ID so that I can establish a persistent, real-time connection to a game instance.

F-1.4 As a System Admin, I want to delete empty or stale rooms via the API so that I can free up server resources when a game session has concluded.

F-1.5 As a player, I want each room to enforce a maximum player limit so that gameplay remains manageable and fair.

F-1.6 As a system, I want to assign player seats automatically upon joining so that table positions are clearly defined.

5.2.2 Game Logic & Rules

F-2.1 As a player, I want to be dealt two private hole cards at the start of a hand so that I can begin assessing my strategy for the round.

F-2.2 As a player, I want to have the system enforce standard Texas Hold'em rules so that I cannot make illegal moves like checking when a bet has already been placed.

F-2.3 As a player, I want to see community cards dealt in the correct stages (Flop, Turn, River) so that the game progresses naturally according to standard poker phases.

F-2.4 As a player, I want to have the pot awarded automatically at showdown or when all others fold so that the winnings are distributed accurately without manual calculation.

F-2.5 As a player, I want to have my chip count updated immediately after betting or winning so that I always know my exact standing in the game.

F-2.6 As a player, I want blinds to be enforced at the start of each hand so that standard poker rules are followed.

F-2.7 As a system, I want to enforce a turn timer (e.g., 30 seconds) so that inactive players do not stall gameplay.

F-2.8 As a system, I want players who exceed the turn timer to automatically fold so that the game continues smoothly.

F-2.9 As a player, I want buy-in limits to be enforced so that all players enter the game with a fair chip range.

5.2.3 User Interface

F-3.1 As a player, I want to see a graphical representation of the table so that I can easily visualize player positions, chips, and the current state of play.

F-3.2 As a player, I want to use action buttons (Fold, Call, Check, Raise) so that I can input my decisions quickly without using text commands.

F-3.3 As a player, I want to use a slider or input box for raising so that I can bet a specific custom amount rather than fixed increments.

F-3.4 As a player, I want to receive visual cues (e.g., highlighting) when it is my turn so that I don't accidentally delay the game by not realizing it is my turn.

F-3.5 As a player, I want to see the current pot size clearly displayed so that I can calculate my pot odds before making a calling decision.

5.2.4 AI Agent Behavior

F-4.1 As a competitive player, I want to play against a Reinforcement Learning Agent so that I can test my skills against an AI that adapts its play style to optimize decisions.

F-4.2 As a competitive player, I want to play against a variety of agent types so that I can have a baseline opponent or a chaotic game experience.

F-4.3 As a competitive player, I want to see the AI bluff occasionally so that the game feels realistic, as the AI isn't just playing based on pure hand strength.

F-4.4 As a developer, I want to load AI models at server startup so that the agents are ready to play immediately without loading delays during the game.

5.2.5 Edge Cases & Error Handling

F-5.1 As a player, I want to reconnect to a game after a disconnect so that I can resume my session without losing progress.

F-5.2 As a system, I want to prevent duplicate player joins in the same room so that each user occupies only one seat.

F-5.3 As a player, I want invalid bets (e.g., negative values or exceeding chip count) to be rejected so that game integrity is maintained.

F-5.4 As a system, I want to resolve simultaneous player actions using server-side ordering so that game state remains consistent.

F-5.5 As a system, I want to handle AI model loading failures gracefully so that fallback behavior or error messages are provided.

F-5.6 As a system admin, I want stale rooms to be automatically removed after a timeout so that server resources are conserved.

5.3 Performance Requirements

P-1 As a player, I want the UI to update within 100 milliseconds of my action so that the game feels responsive.

P-2 As a player, I want AI decisions to be completed within 2 seconds ($p99 \leq 2.5$ seconds) so that gameplay maintains a natural pace.

P-3 As a developer, I want the system to support at least 50 concurrent rooms with up to 6 players each (300 concurrent users) so that multiple games can run simultaneously.

P-4 As a developer, I want WebSocket messages to support a throughput of at least 500 messages per second so that real-time updates scale effectively.

P-5 As a developer, I want the system to maintain $p99$ latency ≤ 200 ms for bet-to-table update events so that all players receive near-simultaneous updates.

P-6 As a developer, I want broadcast latency across all players in a room to remain ≤ 150 ms ($p99$) so that gameplay remains synchronized.

P-7 As a developer, I want the system to remain stable for 100+ consecutive hands without crashes or memory leaks.

P-8 As a system, I want to degrade gracefully under load so that if AI response time exceeds thresholds, a fallback strategy is used.

P-9 As a system, I want to maintain 99.5% weekly uptime so that the application is reliably available to users.

P-10 As a developer, I want the system to pass load testing with 300 concurrent users and $<5\%$ error rate so that performance requirements are validated.

5.4 Environment Requirements

E-1 As a developer, I want the backend to run on Python 3.11+ with FastAPI (v0.110+) so that modern async features are supported.

E-2 As a developer, I want the frontend to use React 18+ with Vite and Node.js 20+ so that the client is optimized for modern web development.

E-3 As a developer, I want the system to use PyTorch 1.9+ so that reinforcement learning models can be supported.

E-4 As a developer, I want the system to use PostgreSQL for persistent storage so that user data and hand history are reliably maintained.

E-5 As a developer, I want the system to use Redis for caching and real-time state management so that performance is optimized.

E-6 As a developer, I want the system to run on a machine with an NVIDIA GPU (≥6GB VRAM, CUDA support) so that AI inference is accelerated.

E-7 As a developer, I want the system to support a CPU-only fallback mode so that gameplay remains functional without a GPU.

E-8 As a developer, I want the system to be deployable using Docker containers so that environments are consistent across development and production.

E-9 As a developer, I want optional deployment support via Kubernetes and cloud platforms (e.g., AWS, GCP) so that the system can scale horizontally.

E-10 As a developer, I want the system to support modern browsers (Chrome, Firefox, Safari – latest 2 versions) so that users can access the game without compatibility issues.

E-11 As a developer, I want to use GitHub Actions and automated testing frameworks (PyTest, Jest) so that builds and tests are consistently validated.

5.5 Glossary

Room: A virtual game table where players join to play a poker session.

Hand: A single round of poker from card dealing to pot distribution.

Showdown: The phase where remaining players reveal cards to determine the winner.

Stale Room: A room that has been inactive for a defined period and is eligible for deletion.

Blinds: Forced bets (small blind and big blind) that initiate action in a hand.

Turn: A player's opportunity to act (fold, call, check, or raise).